Four Zoom Regimes, One Binary: A Load-Balance Characterization of `mandelHybrid`

Matias Saavedra

Friday 10th July, 2026

Binary: `binary-v5-affinity` (`fc33e29`), HEAD `ff1f2f9` (docs only). The four runs share the wide full-set first frame; only the deep-zoom target (`spec.in` line 3) changes. Data: `experiments/13-zoom-points/`.

Resumen

Every prior report measured one zoom target — the canonical minibrot at $c \approx (0.0016, -0.8225)$. This report profiles the current best binary (GPU affinity, report `12-gpu-affinity.tex`) on four qualitatively different targets: fully outside the set, fully inside the main cardioid, and two boundary points distinct from the canonical one. The same wide first frame and `maxIter` ramp are used; only the deep target differs. The headline is a paradox: end-to-end cost spans 32× (2.37 s to 76.3 s) from the same binary and parameters, yet the per-thread CPU spread stays under 1.4 % in every regime — the dynamic queue is not the bottleneck anywhere. The binding resource instead shifts: light regimes are trivially CPU-bound, heavy ones are GPU-saturated (95–96 %). The most useful new finding is the seahorse point, which reproduces the interior outlier eightfold, and all eight are `cardioidBulb`=1 main-cardioid interiors that overflow to the CPU because the GPU is saturated — exactly the case the cheap cardioid certificate would fix, unlike the canonical minibrot (`cardioidBulb`=0). The outliers never bind the wall: the schedule is throughput-bound.

1. Setup and the Four Targets

Workload and machine.

As in reports 09, 11 and 12: `spec.in` of 100 frames at 1920×1080, `maxIterations` ramping $100 \to 10{,}000$, on an i7-9750H (6c/12t) + GTX 1660 Ti Max-Q, 12 threads (1 GPU + 11 CPU), `diffThreshold` $= 0.1$, `pixelSizeThresh` $= 32{,}768$, `save` $= 0$, on AC power. One rep per point (the per-thread spreads below show the within-run balance directly).

The experimental variable.

Every run keeps the identical wide full-set first frame $(-1.9, 0.94)$–$(1.2, -0.94)$ at `maxIter` $= 100$ and changes only the deep target. This makes the zoom target the single independent variable and keeps every run comparable to the others and to the canonical data of prior reports.

Código 1: The four targets – the deep-zoom centre c (spec.in line 3).

```
outside      : c = ( 1.0,         1.0        )   -- exterior, escapes at iter 2
misiurewicz : c = (-0.77568377, 0.13646737)   -- boundary, thin self-similar spiral
seahorse     : c = (-0.745428,    0.113009   )   -- boundary, main-cardioid crossing
inside        : c = (-0.5,          0.0        )   -- interior, period-1, all maxIter
canonical    : c = ( 0.0016437, -0.8224676 )   -- (prior reports) minibrot interior
```

Instrumentation neutrality.

All numbers below are `viz`=0 timing runs. The `viz`=3 animations (next section) were generated separately, because `viz`=3 is not performance-neutral for trivial-region workloads: misiurewicz compute was 2.37 s at `viz`=0 but 66.5 s at `viz`=3, since the per-region `VizRect` registration is a fixed cost that dominates when regions are ∼0.01 ms (it is negligible for the expensive-region points: outside 16.95 → 16.89 s, inside ≈76 s either way). The "performance-neutral" claim of `08-region-metrics-viz.tex` was validated only on expensive-region frames.

2. Results

2.1. The four regimes at a glance

Tabla 1: Four zoom targets, current binary, `diffT`=0.1, 12 threads, GPU on. "Spread" is (max − min)/ max of the eleven CPU-thread wall times. "GPU util" is GPU kernel time / wall. Cost spans 32× but the spread stays under 1.4 % in every row.

Point	Wall (s)	Spread	GPU util	GPU leaves (avg)	CPU leaves	Outliers
outside	16.95	1.4%	79%	2,190 (6.14 ms)	2,758	0
misiurewicz	2.37	0.2%	1.4%	3,312 (0.010 ms)	3,088	0
seahorse	69.82	0.9%	95%	1,128 (58.7 ms)	2,908	8
inside	76.28	0.8%	96%	573 (127 ms)	3,346	0

2.2. Region routing: where affinity sends the work

Tabla 2: GPU share of leaves and mean GPU region size. As the content coarsens toward solid interior, the min-max queue routes fewer, larger regions to the GPU (44 %→15 % of leaves, 6 µs→127 ms each), which is exactly the affinity design at work.

Point	Total leaves	GPU leaves	GPU share	GPU avg (ms)	GPU util
outside	4,948	2,190	44%	6.14	79%
misiurewicz	6,400	3,312	52%	0.010	1.4%
seahorse	4,036	1,128	28%	58.7	95%
inside	3,919	573	15%	127	96%

2.3. The seahorse outliers are all main-cardioid interior

All eight CPU regions exceeding the 5 s `OUTLIER_MS` threshold are 960×540 depth-1 quadrants at frames 85–98, each 17.0–18.7 s, with zero corner spread, 100 % in-set pixels, and — decisively — `cardioidBulb`=1. A representative dump line:

Código 2: lines (experiments/13-zoom-points/logs/seahorse.timing.stderr).]One of the 8 seahorse [OUTLIER] lines (experiments/13-zoom-points/logs/seahorse.timing.stderr).

```
[OUTLIER] frame=98 depth=1 px=960x540 c=[-0.737519,0.110749]..[-0.706519,0.091949]
 corners=9802/9802/9802/9802 spread=0 meanIter=9802.0 inset=100.0% cardioidBulb=1 ms=18689.8
```

This contrasts with the canonical point, whose two binding outliers (frames 73, 89) are `cardioidBulb`=0 minibrot interiors (`08-region-metrics-viz.tex`).

2.4. The partitions, by regime

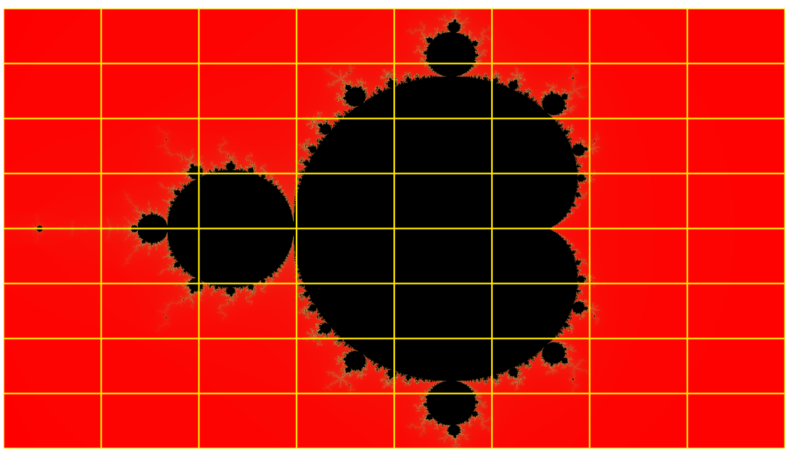

Figura 1: Frame 0, identical for all four runs: the full set at `maxIter`=100. Every leaf outline is yellow (CPU); the GPU does not participate at this cheap wide frame, as `01-initial-profile.tex` noted. This is the common baseline the four targets diverge from.

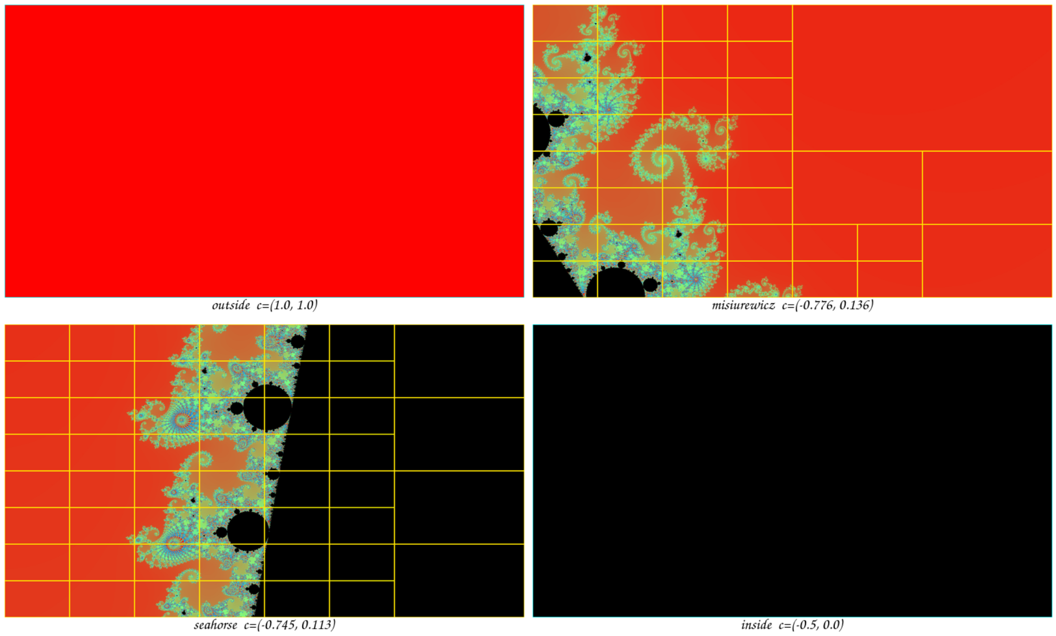

Figura 2: Deepest-frame partitions (`viz`=3). Cyan = GPU leaf, yellow = CPU leaf, grey = split skeleton; colour shows the pixel value (red = fast escape, black = in-set, cyan/green = high-iteration boundary). outside: solid red exterior, a single cyan GPU tile — nothing to balance. misiurewicz: a thin spiral in a sea of red fast-escape exterior — cheap. seahorse: the main-cardioid interior (orange) tiled in yellow CPU leaves (the `cardioidBulb`=1 outliers) beside the spiral and black exterior. inside: solid black interior, a single cyan GPU tile — maximal but uniform work.

3. Analysis

3.1. Cost is content-driven and spans 32× at fixed parameters

The same binary and the same `diffT`=0.1 produce walls from 2.37 s (misiurewicz) to 76.28 s (inside), a 32× range (Table 1). The reason is that total compute is $\sum_{\text{frames}} \sum_{\text{pixels}}$ (iterations to escape or `maxIter`), and the zoom target dictates how much in-set or slow-escape area the path sweeps. The misiurewicz spiral is mostly fast-escape exterior (red in Fig. 2) with only thin filaments, so its 3,312 GPU leaves average 0.010 ms; the inside target is solid interior, so every pixel runs to `maxIter` and its 573 GPU leaves average 127 ms — four orders of magnitude more per region. The implication for the study is that "the workload" is not a single number: any scheduling claim must state which regime it was measured in.

3.2. The dynamic schedule is robust in every regime

The per-thread CPU spread is 0.2–1.4 % across all four points (Table 1) — tighter, even, than the canonical run's 3.8 % (`04-postfix-profile.tex`). The mechanism is the one established in reports 09 and 12: with thousands of fine-grained leaves over 12 threads (∼330–530 leaves per thread here), the tail is always well-filled, so the shared min-max queue keeps every worker busy until the work is gone. This is the central load-balancing result: dynamic balance is already a solved problem on this workload, in every regime tested. The corollary — important for the planned static-load-balancing experiment — is that static decomposition can at best match the dynamic queue here (the headroom is the $<1.4\,\%$ spread), and could only win in a latency-bound configuration the queue is not stressed by.

3.3. The binding resource shifts from the CPU pool to the GPU

The wall is set by a different resource in each regime: GPU utilisation runs 1.4 % (misiurewicz), 79 % (outside), 95 % (seahorse), 96 % (inside) (Table 2). The light regimes are trivially CPU-pool-bound — there is so little GPU-worthy work that thread 0 is idle 98 % of the time. The heavy regimes are GPU-saturated: affinity routes the few large coherent tiles to the GPU (seahorse 28 % of leaves, inside just 15 %, but 58.7 ms and 127 ms each), and the GPU fills to 95–96 %. This is the same $\sim$91 % saturation ceiling report 12 hit on the canonical point, now confirmed to generalise across boundary and interior targets. The implication is that on heavy frames the GPU, not the schedule, is the wall — and the next lever is work-reduction, not scheduling.

3.4. The seahorse outliers: a saturated GPU overflows main-cardioid interior to the CPU

Seahorse is the only regime with outliers — eight of them, all `cardioidBulb`=1 (Table 1, §3.3). They are the largest class of region (960×540, depth 1), which affinity tries to send to the GPU; but the GPU is already 95 % busy with other large tiles, so these eight overflow to CPU threads at 17–18.7 s each. Per-thread balance still holds at 0.9 % because the run is throughput-bound: 8×18 s ≈ 144 s of outlier work overlaps hundreds of other regions across the 69.8 s wall, so no single region binds. The decisive contrast with the canonical point is the membership flag: these are main-cardioid interiors, so unlike the canonical minibrot (`cardioidBulb`=0, on which the cheap test cannot fire) the exact cardioid/bulb certificate of `08-region-metrics-viz.tex` would constant-fill them in $O(1)$. Seahorse is therefore the first concrete, measured beneficiary of that certificate.

3.5. Uniform interior produces zero outliers

Inside is the heaviest run (76.3 s, every pixel to `maxIter`) yet has no outliers (Table 1). The mechanism is two-fold: a uniform interior has zero corner spread everywhere, so the heuristic subdivides only down to the `pixelSizeThresh` floor, producing many equal medium tiles rather than one giant one; and GPU affinity then takes the largest of those (127 ms avg), so the biggest piece any CPU thread ever sees stays under 5 s. Maximal total work does not imply a tail — the tail is a boundary phenomenon (seahorse, canonical), not a volume one.

4. Recommendations

1. Use this as the baseline for the static-load-balancing study. Dynamic balance is <1.4 % spread in all four regimes, so static decomposition (block vs. cyclic vs. block-cyclic) should be measured for parity at lower overhead, not for a wall win — and specifically in a latency-bound corner (CPU-only, or GPU-starved) where a single region can bind the critical path, which never happens here.

2. Implement the cardioid/period-2 bulb certificate; seahorse is its test case. The eight `cardioidBulb`=1 outliers (≈144 s of CPU work) would be constant-filled in O(1). Expected wall effect is bounded by GPU saturation (the relieved CPU work must still beat the 66.2 s GPU floor), so target the CPU-pool-bound and latency-bound configurations where it pays most. It does not help the canonical minibrot (`cardioidBulb`=0).

3. Treat the 95–96 % GPU saturation as a general ceiling. It reproduced on both heavy targets, confirming report 12: past it, only work-reduction (periodicity checking in `diverge()`) shrinks the wall.

4. Fix the filename-prefix buffer (`main.cpp:201`, `char imageFilePrefix[MAXFNAME-8]`, ≈42 B, read by `fin » imageFilePrefix`): prefixes ≥42 chars truncate silently (the `viz_misiurewicz/` overflow seen here). A `char[256]` or `snprintf` bound fixes it; it does not affect any measurement.

5. Conclusion

Profiling the GPU-affinity binary across four zoom regimes shows that end-to-end cost varies 32× (2.37–76.3 s) while the dynamic schedule stays balanced to under 1.4 % everywhere: dynamic load balancing is already solved on this workload, and the binding resource is content, not scheduling. In light regimes the CPU pool binds trivially; in heavy regimes the GPU saturates at 95–96 %, the same ceiling report 12 found, now shown to generalise. The seahorse boundary target reproduces the interior outlier eightfold and identifies it as `cardioidBulb`=1 main-cardioid interior overflowing a saturated GPU — the first measured case the cheap cardioid certificate would relieve, in contrast to the canonical `cardioidBulb`=0 minibrot. The outliers never bind the throughput-bound wall. The result reframes the remaining work: scheduling is exhausted, and the levers are an interior certificate (for boundary tails) and work-reduction (for the GPU-saturated volume).